

课程笔记

CS224R Lecture 10: RL for LLM Reasoning

Codex

2026-04-13



视频作者/频道: Stanford CS224R / guest lecture by Aviral Kumar

发布日期: 2025-05-02

视频时长: 约 01:10:30

视频链接: <https://www.youtube.com/watch?v=02VpNnwB41M>

目录

1	为什么“把推理做成 RL”成了一个核心问题	2
1.1	把数学推理看成一个稀疏奖励 MDP	2
1.2	为什么仅靠模仿不够	3
1.3	本章小结	3
2	离线视角：先识别坏步骤，再决定如何学习	3
2.1	伪步骤为什么会损害泛化	3
2.2	优势函数在这里到底表示什么	4
2.3	离线 RL 怎么用这些优势	5
2.4	本章小结	5
3	在线视角：从 0/1 outcome reward 到 dense process reward	5
3.1	最基础的在线 RL 配方	5
3.2	Process Advantage Verifier: 把稀疏回报变稠密	6
3.3	GRPO 与 rollout-based advantages	7
3.4	本章小结	7
4	Thinking Models 其实仍然站在老配方上	8
4.1	从 token 动作到“元动作”	8
4.2	为什么“更长推理”会成为一个新范式	8
4.3	仍然悬而未决的问题	9
4.4	本章小结	9
5	总结与延伸	9

1 为什么“把推理做成 RL”成了一个核心问题

这节课讨论的不是一般意义上的“让语言模型更会说”，而是更狭义的“让模型在数学与复杂推理任务上更会解题”。讲者先从一个现实前提出发：标准的 next-token prediction 在互联网文本足够丰富时非常有效，但在数学推理、复杂规划、具身决策等场景里，可直接模仿的高质量轨迹远少于普通语言建模数据，而且现成数据还会带有偏差。仅仅扩大 SFT 语料，并不能自动得到稳定的推理能力。

更棘手的是，推理过程并不是只看最终答案的静态映射。模型在中间步骤里可能会出现错误、绕路、伪正确步骤，甚至先犯错再碰巧纠正。若训练只把“最终答对”的整条轨迹当成正样本，那些本应被惩罚的中间步骤也会被一并模仿下来。于是模型在训练题上也许还能靠记忆和局部纠错维持表现，但一旦换到新题，先前学到的坏步骤就会让轨迹偏离。

本讲主线

这节课的技术主线可以概括为三层：先把 LLM 推理解题形式化为稀疏奖励 MDP；再用离线与在线 RL 为步骤分配 credit，避免模仿到伪步骤；最后再回到 DeepSeek-R1 一类 thinking model，说明它们并不是抛弃了经典 RL，而是把同一套 recipe 用在了更长、更具元策略的推理轨迹上。

1.1 把数学推理看成一个稀疏奖励 MDP

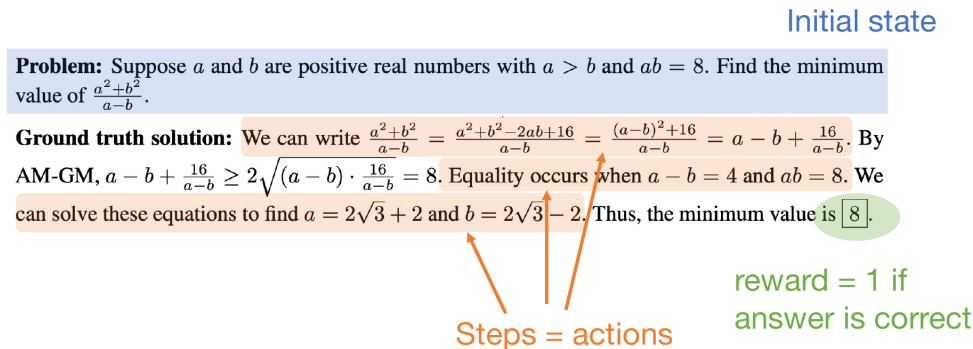
讲者把数学题求解过程写成一个很典型的 RL 问题。初始状态是题目文本与当前上下文；每生成一步中间推理或一个 token 片段，就相当于执行一个动作；直到模型给出最终答案后，系统才根据答案是否正确给予奖励。于是我们可以把奖励写成

$$r(x, y) \in \{0, 1\},$$

其中 x 是题目， y 是整条解题轨迹和最终答案。若最终答案正确，奖励为 1；否则为 0。

这和常见控制任务的区别在于，环境动力学几乎完全由语言前缀拼接决定。生成一步新内容后，下一个状态本质上就是“旧前缀 + 新步骤”。因此它是一个**稀疏奖励、近似确定性动力学**的 MDP：奖励只在终止时给出，而状态转移几乎不依赖外部随机环境。

Problem Setup: LLM Math Reasoning Problems



Intuition: Sparse-reward MDP with deterministic dynamics

图 1: 把 LLM 数学推理问题形式化为稀疏奖励 MDP: 题目为初始状态, 中间步骤是动作, 最终答案正确性给出终止奖励。

这个建模方式的重要意义在于: 一旦推理被写成 MDP, 就可以直接调用 RL 的核心语言来讨论 credit assignment、advantage、offline RL、policy gradient 等问题, 而不必把 reasoning 只理解成“更长的 chain-of-thought”。

1.2 为什么仅靠模仿不够

讲者用两个原因说明“只做模仿”的局限。第一, 高质量 oracle 解答本身稀缺, 尤其在数学、定理证明、复杂规划等领域, 能拿到的大规模专家解轨迹远少于普通文本。第二, 即便从当前模型 on-policy 采样出大量看似可用的解题轨迹, 直接把其中答对的轨迹当作 SFT 数据继续拟合, 也会逐渐伤害泛化。

根本原因是: 答对不代表每一步都值得学。如果某条正确轨迹里包含一个明显错误但后来被模型碰巧纠正的步骤, 那么这个错误步骤仍然会进入训练集。对固定提示集合反复训练后, 模型会在这些已见 prompt 上学会“带着错误也能勉强收回来”, 但在新题上就未必能恢复。

1.3 本章小结

这部分完成了问题设定。课程并不是在讨论“RL 能不能让模型更会聊天”, 而是在讨论: 当推理数据有限、步骤质量参差不齐时, 怎样用 RL 的 credit assignment 语言, 细化地判断哪些步骤该学、哪些步骤不该学。后面所有方法, 都是围绕这个目标展开。

2 离线视角: 先识别坏步骤, 再决定如何学习

2.1 伪步骤为什么会损害泛化

讲者对“spurious steps”的解释非常重要。所谓伪步骤, 并不是指轨迹最终失败, 而是指某些中间步骤本身错误、无关或者会把状态带偏, 只是这条具体轨迹后来又绕回正确答案。这样的步骤在训练数据里看起来无害, 但在测试时会把模型带去错误状态分布, 造成泛化下降。

从 imitation learning 的角度看，这和“在错误历史上继续模仿”的问题很像。模型会在训练分布中见到这些奇怪历史，并学会在其后做出补救；可一旦到了新的题目、不同的错误上下文，这些补救未必成立。

An Idea: Assigning “Credit” to Steps

Question: 4 apples and 3 pears cost \$25, but 7 apples and 6 pears cost \$49. What is the cost of 1 apple?

Key insight: If we can identify spurious steps somewhat *precisely*, then we can avoid this issue

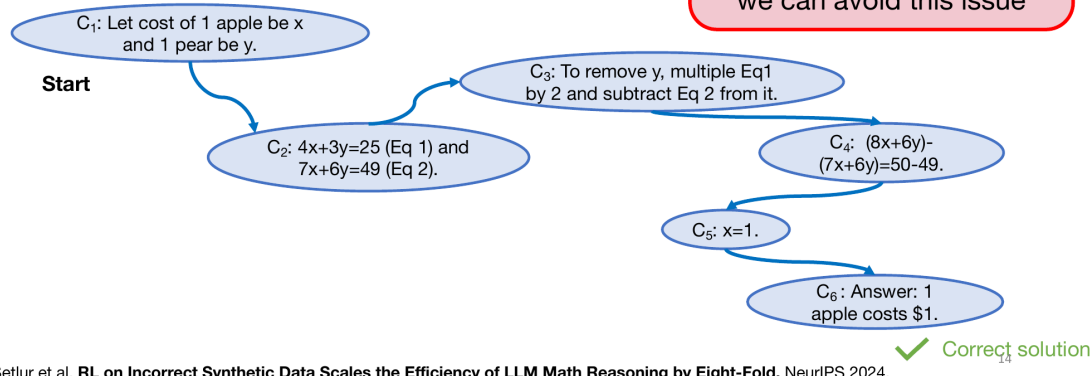


图 2: 通过步骤级 credit assignment 识别坏步骤：最终答对并不意味着每个中间步骤都值得监督。

因此关键不是简单地区分“正确轨迹”与“错误轨迹”，而是要进一步分配**步骤级别**的 credit。讲者用苹果和梨的代数题说明：如果第 4 步把式子写错，即便第 5 步又偶然修正回来，第 4 步仍应视为负贡献。

2.2 优势函数在这里到底表示什么

在标准 RL 中，优势函数写作

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

它表示：在状态 s 下，动作 a 相比该策略平均行为究竟更好还是更差。

而在推理 MDP 里，状态是前缀，动作是当前步骤，所以更自然的写法是对相邻前缀的未来成功概率做差：

$$A_\pi(x, y_{0:i-1}; y_i) = Q_\pi(x, y_{0:i}) - Q_\pi(x, y_{0:i-1}).$$

这里 $Q_\pi(x, y_{0:i})$ 可以理解为：在题目 x 下，当前已经写到前缀 $y_{0:i}$ 时，后续继续 rollout，最终答对的期望概率。于是：

- 若 $A_\pi > 0$ ，说明这一步提高了后续成功率；
- 若 $A_\pi < 0$ ，说明这一步让成功率下降，很可能就是伪步骤；
- 若 $A_\pi \approx 0$ ，则说明这一步几乎没有帮助。

讲者特别强调，在这个确定性前缀 MDP 里，用相邻前缀的 Q 值差代替传统的 $Q - V$ ，本质上仍然是在做 advantage estimation，只是写法更贴合语言生成。

2.3 离线 RL 怎么用这些优势

有了步骤级优势，离线学习就不再需要把整条轨迹一视同仁地当成监督。课程给出两种直觉上非常清晰的做法。

第一种做法是**过滤**：直接丢掉低优势甚至负优势的步骤，只保留那些明确提升后续成功率的局部决策，让训练集从“整条正确轨迹”变为“正确轨迹里的高价值片段”。

第二种做法是**加权保留**：不必彻底删除整段轨迹，而是让高优势步骤在训练目标中占更大权重，低优势或负优势步骤权重更小。这样做更像 advantage-weighted regression 或 DPO 风格的离线 RL。

Using Advantages for Training

Option 1: Filter steps by advantages directly

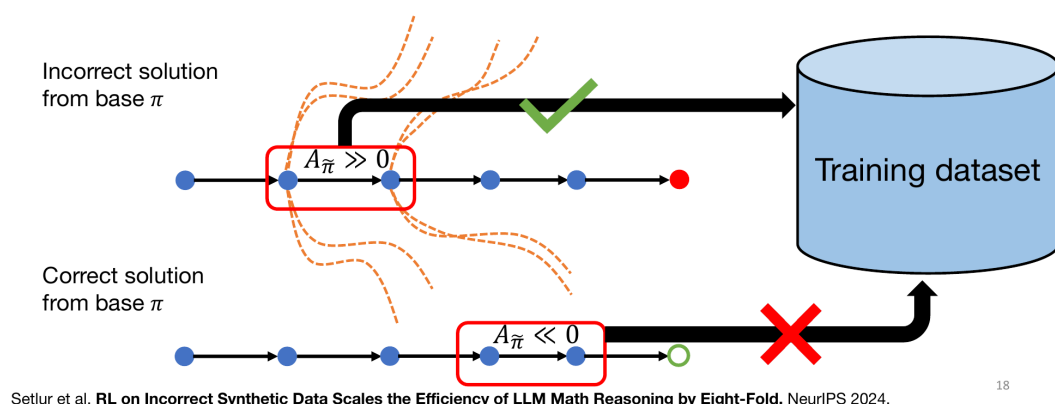


图 3: 用优势直接筛选训练步骤：同一条轨迹中，真正提高成功率的步骤应被强化，负优势步骤应被降权或删除。

讲者报告的核心经验结论是：当训练数据里混入模型自己生成的负样本和坏步骤时，离线 RL 相比单纯 imitation 能显著提高样本效率。换句话说，坏数据并非毫无用处，前提是你能正确地给坏步骤分配负 credit。

2.4 本章小结

这部分的核心收获是：reasoning 训练真正缺的不是更多“答对了”的轨迹，而是一个能指出“哪一步在帮忙、哪一步在捣乱”的评估信号。优势函数把这个问题形式化之后，离线 RL 就能从正负混合轨迹里学到比纯 SFT 更干净的步骤偏好。

3 在线视角：从 0/1 outcome reward 到 dense process reward

3.1 最基础的在线 RL 配方

一旦把推理任务写成 MDP，最朴素的在线 RL 做法就非常直接：从当前策略采样解题轨迹，比较最终答案和标准答案，根据是否正确给出 0/1 回报，再用 policy gradient 优化。

其目标可以写成

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)}[r(x, y)],$$

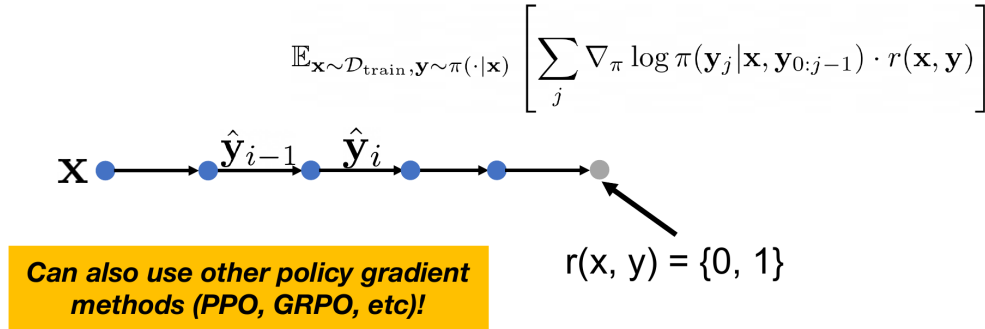
对应的 REINFORCE 形式梯度为

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}} [r(x, y) \nabla_{\theta} \log \pi_{\theta}(y|x)].$$

Online RL: Basic Recipe with 0/1 rewards

A solution from
current π

Key idea: Train with a binary 0/1
reward using policy gradients



Setlur et al. Rewarding Progress: Scaling Automated Process Supervision for LLM Reasoning. arXiv 2024.

24

图 4: 在线 RL 的最基础 recipe: 从当前策略采样整条解题轨迹, 用最终答案正确性给 0/1 奖励, 再用 policy gradient 更新。

这就是许多 reasoning RL 工作的出发点。PPO、GRPO 之类方法, 本质上都是围绕这个目标做更稳健的优化, 而不是改变问题本身。

不过, 讲者紧接着指出, 仅靠 outcome reward 往往过于稀疏。模型只有在整道题结束时才知道自己是否成功, 中间每一步的优劣都必须靠末端回报反推, 这会带来高方差和低样本效率。

3.2 Process Advantage Verifier: 把稀疏回报变稠密

为了解决稀疏奖励问题, 课程引入了 Process Advantage Verifier, PAV。其思想并不是另起炉灶, 而是把刚才离线部分的 advantage 思想迁移到在线 RL 中: 既然可以为每个步骤估计“它对最终成功率的边际贡献”, 那就可以把这个量当成**稠密奖励 bonus**。

Approach: Process Advantage Verifiers (PAVs)

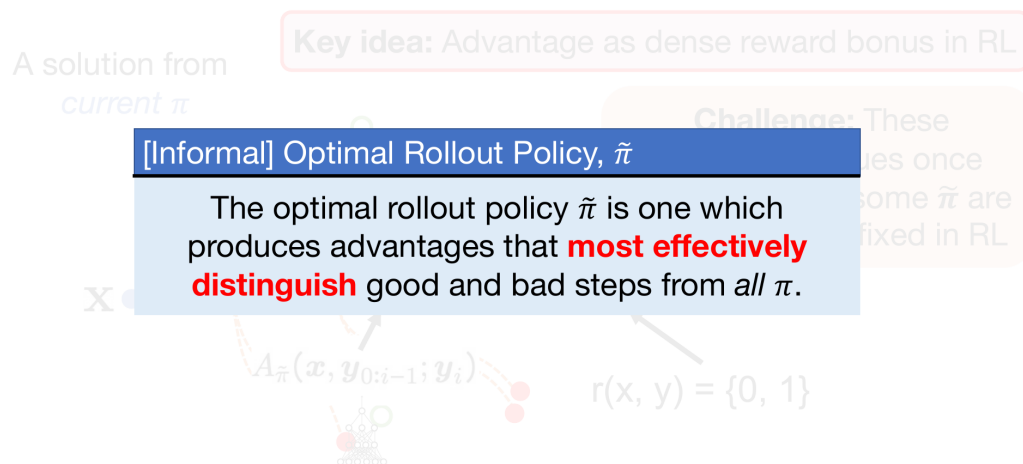


图 5: PAV 的关键直觉：估计步骤级 advantage，把它作为 dense reward 加到在线 RL 训练中。

这样做的好处有两层：

- 在训练信号层面，模型不再只在结尾拿到一个二元回报，而是在每一步都收到“这一步对成功是否有帮助”的反馈；
- 在优化层面，策略梯度不必完全依赖长程 credit propagation，样本效率会明显改善。

讲者也把它和 NLP 社区常说的 PRM, Process Reward Model 联系起来。PRM 可以理解为给中间过程打分的模型，而这里的 advantage estimation 则提供了一个更明确的 RL 语义：它度量的是某一步相对前缀基线的增益或损害。

3.3 GRPO 与 rollout-based advantages

讲者在回答学生提问时强调，GRPO 并不神秘。它可以理解为：把 PPO 中依赖学习到的 value function 的部分，用基于 rollout 的 advantage 估计替换掉，从而更贴合 reasoning 场景里“可以通过多次 rollout 近似评估未来成功率”的事实。

因此，DeepSeek-R1 一类方法与经典 RL 的关系不是断裂，而是延续：

- 仍然在优化同一个推理 MDP；
- 仍然依赖 policy gradient 家族；
- 只是在 advantage 的估计方式、奖励 shaping、轨迹长度和动作语义上做了适配。

3.4 本章小结

在线 RL 的基本 recipe 并不复杂，难点在于奖励过稀。PAV 与类似 PRM 的技术告诉我们，真正有效的方向不是抛弃 RL，而是用更细粒度的过程信用信号，把“最后答对”变成“每一步到底有没有推动答对”。

4 Thinking Models 其实仍然站在老配方上

4.1 从 token 动作到“元动作”

讲者后半段的一个重要判断是：今天的 thinking models 在目标函数层面并没有背离前述 RL 视角，真正变化的是**动作空间的语义**。更强的基础模型能执行更高级的认知行为，比如回看、验证、回溯、设子目标、反向链式推理，这些都可以视作更高层的“macro actions”。

“Action” Space: Incorporating Meta Strategies



This model does not improve during RL

Gandhi et al. **Cognitive Behaviors that Enable Self-Improving Reasoners**. arXiv 2025.

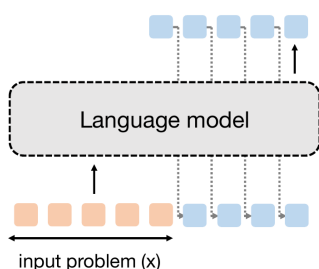
图 6: 更强的 reasoning 模型会在 RL 训练中涌现出 verification、backtracking、subgoal setting 等元策略。

这意味着同一个 RL 目标函数，在不同能力边界的 base model 上会体现出不同的行为形态。基础模型若连基本纠错都不会，RL 只能在低层 token 空间里做局部搜索；而当模型已经具备一定抽象能力，RL 就能放大这些更高层的认知策略。

4.2 为什么“更长推理”会成为一个新范式

讲者把 test-time scaling 看成 thinking models 兴起的重要背景。过去我们往往靠增大参数量、增大训练数据来提升性能；而在 reasoning 任务上，允许模型在测试时消耗更多计算、产生更长推理轨迹、进行自检和重试，本身就可能带来显著收益。

A New Paradigm: Test-Time Scaling



Snell et al. Scaling LLM Test-Time Compute Optimally can be more Effective than Scaling Model Parameters. ICLR 2025 (Oral).

37

图 7: Test-time scaling 的抽象示意: 固定输入问题, 允许模型在测试时投入更多计算与更长推理轨迹。

这并不意味着“只要输出更长就一定更好”。更准确的说法是: **如果 RL 能教会模型更有效地使用额外推理预算, 那么额外 compute 才会真正转化为更高正确率**。所以训练时的奖励设计、步骤评价和元策略发现, 直接决定了测试时额外算力会不会被浪费。

4.3 仍然悬而未决的问题

课程最后没有把 DeepSeek-R1 或其他模型当作终点, 而是提出了若干仍未解决的方向:

- 仅靠 outcome reward 是否足够, 还是必须引入更丰富的 dense reward;
- RL 相比纯 SFT 的优势来自哪里, 是探索、credit assignment、还是更好的 adaptation;
- 如何把 reward signal 设计成既不误导模型, 又足以支持长程推理;
- 当模型越来越会“想”时, 我们究竟是在优化正确性, 还是只是在放大某些表面行为。

4.4 本章小结

thinking model 的流行, 并没有推翻经典 RL for reasoning 的理论框架。相反, 它证明了老问题仍然成立: 如何定义轨迹、如何给步骤分配 credit、如何让额外计算预算转化为有用探索。区别只是今天的动作空间更高阶、轨迹更长、奖励更难设计。

5 总结与延伸

Lecture 10 的真正价值, 在于它把“LLM 推理训练”从流行语拉回到一个严谨的 RL 叙事里。首先, 数学与复杂推理可以被表述成稀疏奖励、确定性前缀动力学的 MDP; 其次, 推理训练的核心矛盾不是缺少正样本, 而是中间步骤质量难以判定, 因此必须做步骤级 credit assignment; 再次, 无论是离线 RL 的优势加权, 还是在线 RL 的 PAV、GRPO, 本质上都在回答同一个问题: 怎样让模型学会真正帮助成功的步骤, 而不是把碰巧成功的轨迹整体背下来。

如果把这节课与当下的 reasoning 模型热潮联系起来, 可以得到一个很清楚的判断: DeepSeek-R1、Kimi 等系统之所以有效, 不是因为它们摆脱了 RL, 而是因为它们把 RL 用到了更合适的对象上。

更强的基础模型、更长的 token budget、更多 rollout, 以及能识别 verification/backtracking/subgoal 价值的奖励机制, 共同构成了今天 reasoning RL 的工程现实。

从研究角度看, 后续最值得追问的方向有三个。第一, 能否构造更可靠的过程奖励, 而不是只在结尾打分。第二, 能否把步骤级 credit assignment 做得更稳健, 避免 reward hacking。第三, 能否把“更长推理”从粗糙的长度扩张, 变成真正高价值的测试时搜索。这三点, 也正是这节 guest lecture 留给 CS224R 后续内容的延伸空间。